Reward Function’s Impact on RL Agents in Traffic Control

Asher M. Usdan
Science, Engineering & Research Expo 2026
Grade 8

Table of Contents

Abstract

This project investigated how reinforcement learning could be used to control traffic lights. Each year in the United States, approximately 40,000 fatalities result from car accidents, with nearly half occurring at intersections. Additionally, cars are responsible for approximately 28% of global greenhouse gas emissions. This project addressed these critical issues by both reducing the number of crashes and the wait time for cars. Across 13 training runs on an NVIDIA GPU, a reinforcement learning (RL) agent learned to control traffic lights, with the implementation comprising over 900 lines of Python code. The hypothesis was that if a well-designed reward function is used, then the RL agent will control the traffic lights effectively, because the reward will guide it toward better decision-making during training. The reward function serves as the feedback to the agent, indicating how well it is doing in the environment. The reward function guides the agent's learning process by assigning numerical values to its actions. To test the hypothesis, two different training runs were compared: the training run that does the best job at traffic control and a training run with a reward function that is purposefully poorly designed. The reward function of the better agent was: reward = -2 * wait time + 50 * cars passed + -10,000 * number of crashes. The reward function of the agent with a poorly designed reward function was: reward = wait time * -10 + cars passed * 500 + number of crashes * -10. The results showed that the RL agent that was trained with a well-designed reward function performed better than the RL agent that was trained with a poorly designed reward function at traffic control. This project demonstrates how scientific testing and engineering design can be used together to fix real-world problems.

Introduction

The purpose of this investigation was to train a reinforcement learning agent to control traffic lights and compare agents trained with different reward functions. This was done by doing over a dozen training runs and recording the average number of crashes and wait time. For each training run, one aspect of how the agent is trained was changed. Examples are: Reward function, observations, possible actions, training length, etc. Each training run took about 30 hours to complete, but some took a longer or shorter amount of time.

The hypothesis for this project was: *If a well-designed reward function is used, then the RL agent will control the traffic lights effectively, because the reward will guide it toward better decision-making during training.*

The independent variable in the investigation was the reward function. The dependent variables were the average number of crashes per car per minute and the average wait time per car per minute. Controlled variables included the car spawn rate, the amount of time per episode (60 seconds), and the values for training, like the batch size and learning rate.

This investigation connects to real-world applications by demonstrating how artificial intelligence could be used in traffic control to improve road safety and reduce pollution.

Background Information

My topic is computer science. I chose it because I have lots of experience with computers, and the judges like complex projects. For my science fair project, I will program a traffic intersection simulation in Python. Then, I will train a reinforcement learning (RL) agent to control the traffic lights to reduce crashes and wait times. According to my research, a reward function determines how the RL agent receives rewards. The agent then uses these rewards to learn if it did well or not. An RL agent is a type of artificial intelligence that learns from trial and error. In my project, I will explore how the design of the reward function affects the agent's ability to control the traffic lights.

A key vocabulary term for this project is reward hacking. According to research, reward hacking refers to a situation in which an artificial intelligence finds a way to maximize its rewards without actually fulfilling the intended goals set by humans.

Another key vocabulary term is reinforcement learning, or RL for short. There are several ways that AI learns, one of which will be used for this project is reinforcement learning. In RL, the agent receives information about the environment called the observation. The agent also has a choice of actions to take. When it's training, it starts out taking random actions, and after each action, it receives a reward from the reward function. A higher reward means that the agent’s action was good, and the agent’s job in training is to adjust its “brain” to produce the action according to the observations that will produce the highest reward.

I learned a lot from my research. I learned how to use the Gymnasium Python library from the Gymnasium documentation website. A library in Python is where you

import someone else's code for a hard task that you wouldn’t be able to do by yourself. The Gymnasium library provides me with functions for making the environment that the RL agent interacts with. Google Maps will be used to see where the lines are supposed to be on the road to make my simulation as realistic as possible. I used various websites to research how to use the Pygame library, which helps with making visualizations on the screen. I also watched videos on how machine learning and neural networks work. Finally, I used multiple websites to research how to use some Python functions I wasn’t familiar with.

I will use the RTX 4000 virtual computer from a website called Paperspace to train the RL agent. My personal computer doesn’t have enough computing power to train it, so using the virtual machine will be very helpful.

In conclusion, I conducted lots of research. I learned how to utilize various functions and libraries in Python that I will use in my project. I also learned what reward hacking is, I know how reinforcement learning works, and I will use a virtual machine for training the AI. So far, I have a plan for my project.

Materials

- Computer
- Visual Studio Code
- Virtual RTX 4000 Processor (via Paperspace)
- Charger

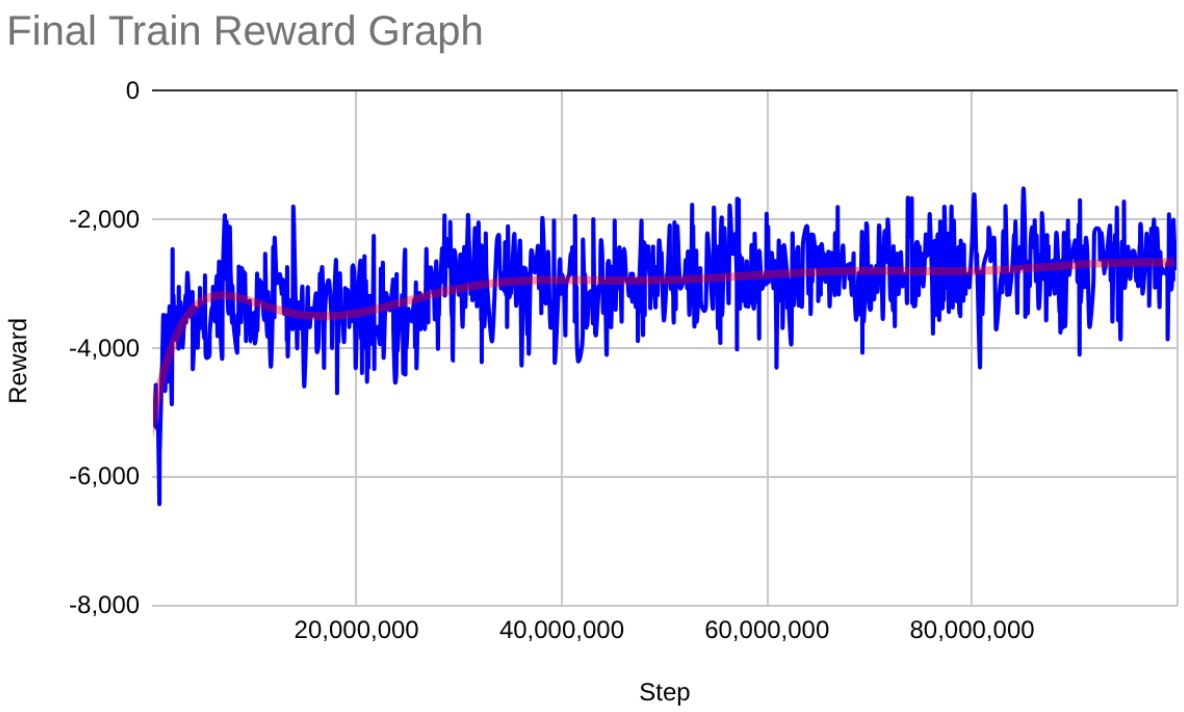
Procedure

1. Start up the virtual machine on the Paperspace website
2. Open up VS Code with remote SSH and connect with the virtual machine
3. Enter “python3 traffic_train.py” in the terminal to start training the RL agent
4. Leave the computer on overnight to train
5. When it's done training, download the model and normalized environment files to my computer
6. Enter “python traffic_run.py” to test the agent on the simulation
7. Take a video of the agent controlling the traffic lights for the presentation
8. Enter “tensorboard --logdir traffic_logs” and record the ep_reward_mean graph–the average reward for the agent on each episode–in a spreadsheet
9. Enter “python get_averages.py” to get the average crashes and wait time per minute, and record them in a spreadsheet.
10. If it isn’t controlling the traffic lights properly, change a variable and go to step 3.
11. Change the reward function to a poorly designed reward function to demonstrate reward hacking and do steps 3-9.

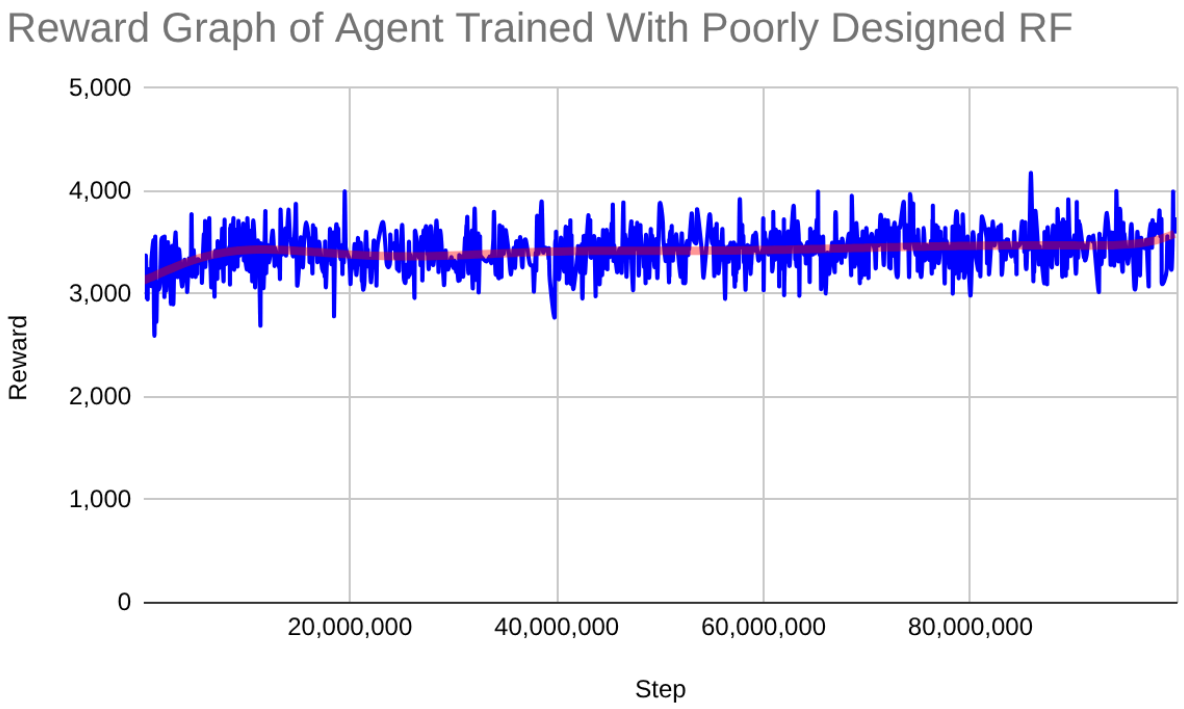
Results

For readability's sake, the RL agent trained with a well-designed reward function will be called agent A, and the agent trained with a poorly designed reward function will be called agent B. The collected data showed a clear difference between agent A and agent B. Both agents were trained for the same amount of time in the same environment with the same observations and possible actions, which ensured that the comparison was fair and controlled. As time progressed, the reward quickly grew for agent A, and there was little to no change in reward for agent B.

This is the reward graph for agent A:



This is the reward graph for agent B:



The reward for agent A went from about -6,000 to -3,000, and the reward for agent B went from around 3,000 to about 3,500. An inference made by looking at the charts could be that agent B performed better at controlling traffic lights because the reward is so much higher, but this inference is incorrect. This is a chart of the average number of crashes and wait time for Agents A and B:

Table 1: Average Crashes and Wait Time for Agents A and B

	Agent A	**Agent B**
Average number of crashes per minute	1.8	4.2
Average wait time per car	3.6	5.5

The chart clearly shows that agent A performed better than agent B at controlling traffic lights because there are, on average, fewer crashes and less wait time per car. These are videos of agents A and B controlling traffic lights:

Video of Agent A

Recording 2026-01-02 093732.mp4

Video of Agent B

Recording 2026-01-11 123312.mp4

As shown in the videos, agent A clearly outperformed agent B. This is an example of reward hacking because agent B learned to maximize the reward without fulfilling the task it was meant to do.

Design Process

12 training runs were conducted on the RTX 4000 GPU that was rented from Paperspace for $0.50 per hour when running and $0.01 per hour when not running. The RTX 4000 was chosen because it has 8 CPU cores and a high-speed GPU for a relatively low price. This is a link to a copy of the project folder so judges can look at the code: Science Fair 2025-2026

traffic_sim.py is the Python file storing the code for the simulation of the intersection. It contains a car class, used for creating and updating the cars, and a sim class used for creating a simulation of an intersection.

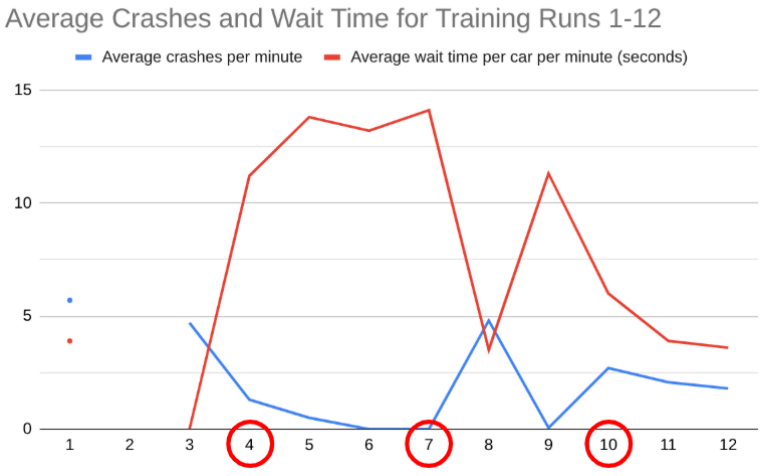
traffic_env.py is the Python file storing the code for the gymnasium environment that the RL agent will interact with. Gymnasium is a useful Python library for RL projects. Inside traffic_env.py is a Python class called TrafficEnv. The RL agent will call the step function of the TrafficEnv with its action as a parameter, and receive observations and a reward.

traffic_train.py is the Python file that contains the code for training an RL agent. It uses the Python library stable_baselines3 for training. The reinforcement learning algorithm used is called proximal policy optimization, or PPO. PPO is chosen because it is a widely used, stable, and simple algorithm. Another reason for using PPO is its ability to remember previous actions when updating the neural network. This is important because, if there is a car crash in the simulation, the bad decision could have been made hundreds of actions before the crash.

When traffic_run.py is run, it makes a window of the simulation, and every frame, it runs the step function in traffic_env.py. The agent gets the observations and takes an action. traffic_run.py was used to make the videos of the simulation.

get_averages.py is for getting the average crashes and wait time for each agent. It is similar to traffic_run.py except it doesn't render the simulation on the screen. When get_averages.py is run, it runs the simulation 15 times and prints the average crashes and wait time.

After each training run, a screen recording was made of the agent controlling traffic lights, which can be found in the data folder of the project folder. The number of crashes and wait time is recorded on a spreadsheet. Then, I watch the video and see what can be improved. For example, if the agent is making cars wait too long, I increase the penalty for crashes. This is a chart of the average crashes and wait time for each of the training runs:



4	Changed to only 3 actions: 1: Horizontal green Vertical red 2: Horizontal red Vertical green 3: Both yellow
7	Made the episode end when there's a crash
10	Built-in cycle. Changed action to how long to hold the light state. Options: 1, 2, 4, 6, 8, … 20 seconds

On the left is a chart where the x-axis is the training run number and the y-axis is the average crashes and wait time per minute. On the right is a chart of three training

runs where a significant change was made. Those specific runs are circled at the bottom of the chart.

Conclusion

The results of this investigation supported the hypothesis that an RL agent trained with a well-designed reward function would control traffic lights more effectively than an RL agent trained with a poorly designed reward function. The agent trained with a well-designed reward function had, on average, a lower waiting time and fewer crashes than the agent trained with a poorly designed reward function.

Both the videos and the charts clearly showed this difference in performance. By presenting quantitative data from the chart and qualitative data from the videos, the hypothesis was clearly supported.

One limitation of this investigation is that the cars in the simulation don’t turn and only move straight. The reason for this is that it adds unneeded complexity to the project. The timeframe for this project was already short, and adding the ability for cars to turn would take too long. Plus, this is just a prototype. This project serves as an example of what traffic lights could look like in the future.

Future research could expand on this project by conducting additional training runs to further improve the RL agent's ability to control traffic lights. Additional investigations could also involve adding the ability for cars to turn and to give the cars a realistic reaction time, like in real life

Understanding artificial intelligence is important because it automates complex tasks, uncovers hidden patterns in massive datasets, and accelerates discoveries.

References

Farma Foundation. (2025, September 23). Gymnasium documentation. Farma. Retrieved October 28, 2025, from https://gymnasium.farama.org/

5801 Forbes Ave. (2025, April 1). Google Maps. Retrieved October 10, 2025, from https://www.google.com/maps/

Martin, R. (2021, December 29). How do I use keyboard inputs in pygame? Stack Overflow. Retrieved October 10, 2025, from https://stackoverflow.com/questions/70523769/how-do-i-use-keyboard-inputs-in-pygame/

Mehmood, S. (2023, January 30). Pygame clock.tick(). Delft Stack. Retrieved October 17, 2025, from https://www.delftstack.com/howto/python-pygame/pygame-clock-tick/

Pygame. (n.d.). Pygame documentation. Pygame. Retrieved October 28, 2025, from https://www.pygame.org/docs/

Python | display text to PyGame window. (n.d.). Geeks for Geeks. Retrieved October 17, 2025, from https://www.geeksforgeeks.org/python/python-display-text-to-pygame-window/

Sanderson, G. (2017, October 17). But what is a neural network? [Video]. Youtube. https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

Sentdex, H. (2020, October 17). Neural networks from scratch in Python [Video]. Youtube.

https://www.youtube.com/playlist?list=PLQVvvaa0QuDcjD5BAw2DxE6OF2tius3V3

Singh, M. (2025, May 2). How to find the length of a list in python. Geeks for Geeks. Retrieved October 10, 2025, from https://www.geeksforgeeks.org/python/python-ways-to-find-length-of-list/

Singh, M. (2025, July 11). Python | getattr() method. Geeks for Geeks. Retrieved October 17, 2025, from https://www.geeksforgeeks.org/python/python-getattr-method/

Singh, T. (2025, October 4). Python pass statement. Geeks for Geeks. Retrieved October 17, 2025, from https://www.geeksforgeeks.org/python/python-pass-statement/

Tripathi, A. (2025, July 23). How to comment out a block of code in python? Geeks for Geeks. Retrieved October 17, 2025, from https://www.geeksforgeeks.org/python/how-to-comment-out-a-block-of-code-in-python/